

Quest 5 Tier Standalone 전환 작업 보고서

작성일: 2026-09-19

작업 브랜치: `Q5Standalone\_Work`

작업 위치: `/Users/healer/Documents/GitHub/ZBReactArchitecture/Joshua\_MAINCODE-works/rees/Q5Standalone\_Work`

1. 작업 목표

이번 작업의 목표는 `module/quest5Tier/ui`를 기존 Module Federation remote 방식에서 Q3처럼 독립 가능한 standalone Vite 애플리케이션으로 전환하는 것이다.

기존 Q5 UI는 portal 또는 host가 remote module을 불러오는 구조에 가깝게 남아 있었다.

```
```text
Vite Federation remote
-> remoteEntry.js
-> exposes ./AppRoutes
```
```

전환 후 목표 구조는 다음과 같다.

```
```text
http://localhost:5185/quest5Tier
-> Q5 standalone Vite app
-> MSAL login
-> BrowserRouter
-> Q5 routes
-> Q5 Functions API http://localhost:7075
```
```

Q3와 다른 중요한 점은 Q5 backend가 Service Bus 기반 command flow를 가진다는 것이다.

```
```text
Q3:
UI -> Functions HTTP -> PostgreSQL
```
```

```
Q5:
UI -> Functions HTTP command endpoint -> Service Bus queue -> queue trigger -> PostgreSQL
```
```

따라서 이번 PR의 핵심 범위는 UI standalone 전환이며, Service Bus local emulator 구성은 별도 논의

### ## 2. 주요 변경 요약

이번 작업에서 구현한 핵심 내용은 다음과 같다.

- Q5 UI를 standalone Vite app으로 전환했다.
- `main.tsx`를 추가해 React root, `AuthProvider`, `BrowserRouter`, `AppRoutes`를 직접 mount하도록 했다.
- Q3 standalone과 같은 MSAL 인증 구조를 Q5에 추가했다.
- `/login` route와 `/quest5Tier` 보호 route를 추가했다.
- 기존 route screen 파일들은 재사용했다.
- 기존 Module Federation 설정을 제거했다.
- React Router framework용 script/dependency를 제거했다.
- 더 이상 사용하지 않는 `routes.ts`를 삭제했다.
- `QUEST5TIER\_UI` local example 값을 backend 포트 `7075`가 아니라 UI 포트 `5185/quest5Tier`로 수정했다.
- loader 실패 시 백지 화면 대신 최소 error message를 보여주도록 했다.

### ## 3. 파일별 상세 설명

### ### 3.1 `module/quest5Tier/ui/main.tsx`

새로 추가한 standalone entry 파일이다.

역할:

- `#root`에 React app을 mount한다.
- `AuthProvider`를 최상단에 감싼다.
- `BrowserRouter`를 standalone app 안에서 직접 제공한다.
- `AppRoutes`를 렌더링한다.

왜 필요한가:

기존 remote 구조에서는 host가 Q5 route module을 가져다 쓰는 형태였지만, standalone app에서는 Q5 자신의 root, router, auth provider를 직접 가져야 한다.

추가된 구조:

```
````tsx
createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <AuthProvider msalInstance={msalInstance}>
      <BrowserRouter>
        <AppRoutes />
      </BrowserRouter>
    </AuthProvider>
  </StrictMode>
);
```

3.2 `module/quest5Tier/ui/auth/msalInstance.ts`

새로 추가한 MSAL client 설정 파일이다.

역할:

- Azure AD client id, tenant id를 설정한다.
- Q3 standalone과 같은 `api://{clientId}/access\_as\_user` scope를 사용한다.
- MSAL cache를 `localStorage`로 설정한다.

왜 필요한가:

Q5 standalone app이 직접 로그인하고 API token을 받아야 하기 때문이다. 기존 remote 구조에서는 auth를 host 쪽에 의존할 수 있었지만, standalone 구조에서는 Q5 UI가 직접 인증 상태를 관리해야 한다.

3.3 `module/quest5Tier/ui/auth/AuthProvider.tsx`

새로 추가한 Q5 auth provider다.

역할:

- MSAL account 상태를 읽는다.
- active account를 설정한다.
- access token을 silent acquire 한다.
- token을 `localStorage.appToken`에 저장한다.
- profile oid를 `localStorage.profileId`에 저장한다.
- 로그인 함수를 제공한다.

왜 필요한가:

Q5 API 호출은 shared `jwtFetch`를 통해 token을 사용한다. `jwtFetch`가 정상 동작하려면 login 이

Token`과 `profileId`가 localStorage에 있어야 한다.

Q5에 맞게 바꾼 부분:

- error log 문구를 `Quest 5 API token`으로 변경했다.
- Q3 코드 패턴은 재사용하되 Q5 standalone app에서 직접 사용할 수 있게 연결했다.

Profile 처리:

`AuthProvider.tsx`에서 로그인 후 MSAL account의 `idTokenClaims.oid`를 `localStorage.profileId`에 저장한다.

```
```ts
localStorage.setItem("profileId", currentAccount.idTokenClaims?.oid ?? "");
```

왜 필요한가:

Q5의 기존 route/API 코드는 로그인된 사용자의 profile id를 `localStorage.profileId`에서 읽는다. 질문 목록 조회, 답변 생성, follow-up 등에서 이 값을 사용한다. standalone으로 바뀌면 host portal의 state에 기대기 어렵기 때문에 Q5 UI 내부에서 같은 값을 직접 저장해야 한다.

Share Question 관련:

`module/quest5Tier/ui/routes/ShareQuestion.tsx` 자체 로직은 새로 만들지 않고 기존 흐름을 유지

기존 흐름:

```
```text
ShareQuestion.tsx
-> getProfileList()로 공유 대상 profile 목록 조회
-> getQuestionById(id)로 현재 question detail 조회
-> question.profileId를 sender profileId로 state에 저장
-> 선택한 receiver profile id 목록과 함께 shareQuestion API 호출
```

즉 share 화면에서 사용하는 profile 값은 두 종류다.

```
```text
localStorage.profileId
 로그인한 현재 사용자 id. Q5 route/API 전반에서 사용한다.
```

```
question.profileId
 공유하려는 question의 owner/sender id. ShareQuestion에서 share command payload에 들어간다.
```

이번 PR에서 바꾼 부분은 `ShareQuestion.tsx`의 공유 로직이 아니라, standalone 로그인 이후 기존 코드 `localStorage.profileId`가 정상적으로 생기게 한 것이다.

### ### 3.4 `module/quest5Tier/ui/routes/Login.tsx`

새로 추가한 login route 화면이다.

역할:

- 로그인 전 사용자를 Microsoft sign-in으로 보낸다.
- 로그인 완료 후 원래 가려던 route로 돌려보낸다.
- 잘못된 redirect path를 막기 위해 `safeReturnPath`를 사용한다.
- 기본 fallback path는 `/quest5Tier`다.

왜 필요한가:

standalone app은 `/login` route를 직접 가져야 한다. 로그인하지 않은 사용자가 `/quest5Tier`에 접근하려 할 때 `requireAuth`가 `/login`으로 redirect한다.

Q5에 맞게 바꾼 부분:

- 화면 title을 `Quest 5 Tier`로 설정했다.
- 기본 return path를 `/quest5Tier`로 설정했다.

### ### 3.5 `module/quest5Tier/ui/AppRoutes.tsx`

기존 route adapter 파일을 standalone app에 맞게 수정했다.

변경 내용:

- `/login` route 추가
- `RequireAuth` wrapper 추가
- `/` 접근 시 `/quest5Tier`로 redirect
- `/quest5Tier` 아래에 기존 Q5 화면 route들을 배치
- wildcard route를 `/quest5Tier`로 redirect
- loader 실패 시 error UI 표시

기존 화면 컴포넌트는 유지했다.

기존:

```
```text
index route
add
:id
:id/add
:id/answer
:id/followUp
:id/edit
:id/share
```
```

변경 후:

```
```text
/login
/
/quest5Tier
/quest5Tier/add
/quest5Tier/:id
/quest5Tier/:id/add
/quest5Tier/:id/answer
/quest5Tier/:id/followUp
/quest5Tier/:id/edit
/quest5Tier/:id/share
*
```
```

왜 필요한가:

remote module일 때는 route가 host 아래에 붙는 전제였지만, standalone app에서는 Q5가 자기 base auth guard를 직접 관리해야 한다.

loader error handling을 추가한 이유:

로컬 검증 중 backend, env, CORS, DB 설정이 맞지 않을 때 route loader가 throw하면 화면이 완전히 다. 그래서 최소한 다음 문구를 보여주도록 했다.

```
```text
Quest 5 data could not be loaded. Please check the Q5 API and try again.
```

이 변경은 standalone 필수 기능은 아니지만, Q5를 독립 실행할 때 실제 문제를 사용자에게 보여주는 최소한의

```
### 3.6 `module/quest5Tier/ui/index.html`
```

Vite standalone entry에 맞게 수정했다.

기존:

```
```html
<title>quest5Tier remote</title>
<script type="module" src="./AppRoutes.tsx"></script>
```

변경 후:

```
```html
<title>Quest 5 Tier</title>
<div id="root"></div>
<script type="module" src="./main.tsx"></script>
```

왜 필요한가:

standalone React app은 `main.tsx`에서 root를 mount해야 한다. `AppRoutes.tsx`는 route definition이고, 더 이상 HTML entry point가 아니다.

```
### 3.7 `module/quest5Tier/ui/package.json`
```

Q5 UI script와 dependencies를 standalone Vite 기준으로 정리했다.

기존 script:

```
```json
"build": "cross-env NODE_ENV=production react-router build",
"build:federation": "vite build",
"dev": "react-router dev",
"dev:federation": "vite",
"preview:federation": "vite preview --port 5185 --strictPort",
"start": "cross-env NODE_ENV=production react-router-serve ./build/server/index.js",
"typecheck": "react-router typegen && tsc"
```

변경 후:

```
```json
"dev": "vite --port 5185 --strictPort",
"build": "vite build",
"preview": "vite preview --port 5185 --strictPort"
```

추가 dependency:

```
```text
@azure/msal-browser
@azure/msal-react
```

제거 dependency:

```
```text
@react-router/node
@react-router/serve
isbot
@originjs/vite-plugin-federation
@react-router/dev
cross-env
```
```

왜 필요한가:

Q5가 더 이상 React Router framework build 또는 Federation remote build를 사용하지 않기 때문에 standalone Vite app에는 Vite dev/build/preview만 있으면 충분하다.

### 3.8 `module/quest5Tier/ui/vite.config.ts`

Module Federation plugin 설정을 제거했다.

기존에는 다음 설정이 있었다.

```
```text
@originjs/vite-plugin-federation
name: quest5TierRemote
filename: remoteEntry.js
exposes: ./AppRoutes
shared: react, react-dom, react-router, react-router-dom
```
```

변경 후에는 기본 Vite 설정만 남겼다.

남긴 것:

- JSX automatic workaround
- build target 설정
- shared-ui alias
- dev server port `5185`

왜 필요한가:

Q5를 remote로 expose하지 않고 독립 앱으로 실행하기 때문이다. 더 이상 `remoteEntry.js`가 필요 없다.

중요:

`vite.config.ts` 파일을 삭제한 것이 아니라 Federation 관련 설정만 제거했다.

### 3.9 `module/quest5Tier/ui/routes.ts`

삭제했다.

기존 내용:

```
```text
@react-router/dev/routes 기반 route config
protectedRoutes export
publicRoutes export
```
```

왜 삭제했는가:

이 파일은 React Router framework용 설정이다. 이번 작업에서 framework scripts를 제거했고, stand

route 정의는 `AppRoutes.tsx` 안으로 옮겼다.

즉 변경 후에는 이 파일을 읽는 consumer가 없다.

Q3에는 비슷한 stale file이 남아 있지만, Q5에서는 새로 작업하는 김에 죽은 파일을 남기지 않는 것이 더 낫다.

### ### 3.10 `module/quest5Tier/ui/env.json.local.example`

새로 추가한 Q5 UI local env example이다.

중요 값:

```
```json
"QUEST5TIER_DOMAIN": "http://localhost:7075",
"QUEST5TIER_UI": "http://localhost:5185/quest5Tier"
```

왜 필요한가:

Q5 standalone app을 로컬에서 실행할 때 `/env.json`에 들어갈 값의 기준이 필요하다.

구분:

```
```text
QUEST5TIER_DOMAIN = Q5 backend Functions API, 7075
QUEST5TIER_UI = Q5 frontend standalone UI, 5185/quest5Tier
```

### ### 3.11 `ui/env.json.local.example`

Portal local env example에서 `QUEST5TIER\_UI` 값을 수정했다.

기존:

```
```json
"QUEST5TIER_UI": "http://localhost:7075"
```

변경 후:

```
```json
"QUEST5TIER_UI": "http://localhost:5185/quest5Tier"
```

왜 필요한가:

Portal HomePage의 Quest 5 card는 `QUEST5TIER\_UI`를 보고 Q5 화면으로 이동한다. `7075`는 backend API 포트이므로 UI 이동 주소로 잘못된 값이다.

### ### 3.12 `module/quest3Tier/ui/env.json.local.example`

Q3 standalone env example에서도 `QUEST5TIER\_UI` 값을 같은 방식으로 수정했다.

왜 필요한가:

각 standalone module이 같은 config shape를 공유하고 있고, cross-module UI URL 예시가 최신 상태이다. Q5 UI가 standalone으로 바뀌었으므로 Q3 local example 안의 Q5 UI 주소도 backend 포트가 아닌 frontend 포트로 맞췄다.

### ### 3.13 `pnpm-lock.yaml`

Q5 UI package 변경에 맞게 lockfile이 갱신되었다.

반영된 내용:

- Q5 UI에 MSAL packages 추가
- Q5 UI에서 Federation plugin 제거
- Q5 UI에서 React Router framework packages 제거
- Q5 UI에서 `cross-env`, `isbot` 제거

왜 필요한가:

`package.json` dependency 변경이 있으므로 lockfile도 함께 업데이트되어야 한다.

## 4. 로컬 검증 중 발견한 문제와 조치

### 4.1 `/env.json`이 HTML을 반환한 문제

증상:

```
```text
Unexpected token '<', "<!doctype "... is not valid JSON
```
```

원인:

`http://localhost:5185/env.json`이 실제 JSON이 아니라 Vite fallback `index.html`을 반환했

영향:

Q5 UI가 `QUEST5TIER\_DOMAIN`을 읽지 못했고, API를 `7075`가 아닌 `5185/questionQry/...`로 호

로컬 조치:

```
```text
module/quest5Tier/ui/dist/env.json
```
```

을 생성했다.

주의:

이 파일은 `.gitignore` 대상인 local runtime file이다. PR에 포함하면 안 된다.

### 4.2 CORS 문제

증상:

```
```text
Access-Control-Allow-Origin header cannot be wildcard '*' when credentials mode
is 'include'
```
```

원인:

Q5 `local.settings.json`의 CORS 값이 `\*`였고, `jwtFetch`는 credentials 포함 요청을 보냈다.

로컬 조치:

```
```json
{
  "Host": {
    "LocalHttpPort": 7075,
    "CORS": "http://localhost:5185",
  }
}
```

```
"CORSCredentials": true
},...
```

주의:

`module/quest5Tier/func/local.settings.json`은 ignored local file이다. PR에 포함하지 않는다.

4.3 Functions 실행 문제

문제:

```
```bash
pnpm exec func start
```

를 실행하면 repo dependency의 오래된 Azure Functions Core Tools `4.4.0`을 사용했고 host가 멈

정상 실행:

```
```bash
cd module/quest5Tier/func
func start --javascript
```

시스템 `func` 버전은 `4.14.0`이었고 정상적으로 route 목록이 출력되었다.

4.4 DB role 문제

증상:

```
```text
Function failed: role "root" does not exist
```

원인:

Q5 local setup fallback이 다음 값을 넣었다.

```
```text
DB_USERNAME=root
DB_PASSWORD=DatabasePassword123!
```

하지만 현재 로컬 PostgreSQL에는 `root` role이 없고, 기존 local convention은 `vscode/vscode`

로컬 조치:

```
```text
DB_USERNAME=vscode
DB_PASSWORD=vscode
DB_HOST=127.0.0.1
DB_DATABASE=quest5Tier
```

### ### 4.5 `quest5Tier` DB 생성 및 마이그레이션

DB가 없어서 다음 명령으로 생성했다.

```
```bash
psql -d postgres -c 'create database "quest5Tier" owner vscode;'
```

```

마이그레이션:

```
```bash
cd module/quest5Tier/func
node deploy/db/updateDbSchemaLocal.js
```

결과:

```
```text
All migrations performed successfully
```

주의:

이 작업은 로컬 DB 상태를 만든 것이며 PR 코드 변경이 아니다.

## 5. 현재 PR에서 빼야 하는 파일

현재 diff에는 Q5 standalone UI와 무관한 파일이 섞여 있다. PR 전 반드시 원복해야 한다.

### 5.1 `deploy/localDev/serviceBus/config.json`

이 파일은 Service Bus emulator queue 설정이다.

현재 변경은 Q5 standalone UI 전환과 직접 관계가 없다. Service Bus local setup을 별도 작업으로 존재 확인이 필요하다.

PR에서 제외해야 한다.

### 5.2 `module/quest5Tier/func/deploy/localDev/serviceBus/queue.json`

이 파일도 Q5 local Service Bus queue 설정이다.

이번 PR이 Service Bus emulator setup PR이 아니라면 제외해야 한다.

### 5.3 `module/profile/func/local.settings.json`

Profile function local CORS 설정 변경이다.

이것은 로컬 설정 파일이고 Q5 standalone 코드 변경이 아니다.

PR에서 제외해야 한다.

## 6. PR에 들어가야 하는 파일

다음 파일들은 Q5 standalone UI 전환에 필요한 변경이다.

```
```text
module/quest5Tier/ui/AppRoutes.tsx
module/quest5Tier/ui/main.tsx
module/quest5Tier/ui/auth/AuthProvider.tsx
module/quest5Tier/ui/auth/msalInstance.ts
module/quest5Tier/ui/routes/Login.tsx
module/quest5Tier/ui/env.json.local.example
module/quest5Tier/ui/index.html
module/quest5Tier/ui/package.json
module/quest5Tier/ui/vite.config.ts
module/quest5Tier/ui/routes.ts 삭제
ui/env.json.local.example
```

```
module/quest3Tier/ui/env.json.local.example
pnpm-lock.yaml
```

7. PR에 들어가면 안 되는 파일

다음 파일들은 local-only 또는 보고서/산출물이다.

```
```text
module/quest5Tier/ui/dist/env.json
module/quest5Tier/func/local.settings.json
module/profile/func/local.settings.json
deploy/localDev/serviceBus/config.json
module/quest5Tier/func/deploy/localDev/serviceBus/queue.json
azurite/
reports/
*.pdf
```
```

8. 검증 결과

8.1 UI build

실행:

```
```bash
cd module/quest5Tier/ui
pnpm run build
```
```

결과:

```
```text
passed
```
```

8.2 UI preview

확인한 URL:

```
```text
http://localhost:5185/quest5Tier
http://localhost:5185/env.json
```
```

`/env.json`은 local ignored file을 넣은 뒤 JSON으로 정상 응답했다.

8.3 Functions

정상 실행 command:

```
```bash
cd module/quest5Tier/func
func start --javascript
```
```

확인한 URL:

```
```text
http://localhost:7075/swagger.json
```
```

route 목록도 출력되었다.

8.4 PostgreSQL

확인:

```
```bash
PGPASSWORD=vscode psql -h 127.0.0.1 -U vscode -d quest5Tier -c 'select current_u
ser, current_database();'
```

결과:

```
```text
current_user | current_database
vscode       | quest5Tier
```

9. Service Bus / Docker 관련 판단

Q5 command flow는 Service Bus를 거친다.

```
```text
UI
-> Functions HTTP command endpoint
-> Service Bus queue
-> queue trigger
-> PostgreSQL
```

따라서 다음 기능은 Service Bus가 없으면 end-to-end로 검증하기 어렵다.

```
```text
create question
update question
create answer
share question
follow-up
```

다만 이번 PR은 Q5 standalone UI 전환 PR이다. Service Bus emulator Docker setup은 별도 범위의
것이 좋다.

Jake에게 확인할 질문:

```
```text
Q3 only needed local PostgreSQL, so no Docker was needed.
Q5 seems different because commands go through Service Bus queues.
Should Q5 local testing use a Docker-based Service Bus emulator, an Azure Servic
e Bus dev namespace, or should command flows be verified elsewhere?
```

### ## 10. PR 설명 초안

```
```text
Convert Quest 5 Tier UI to standalone Vite app
```

- Converted Q5 UI from Module Federation remote to standalone Vite app.
- Added standalone MSAL auth provider, login route, and BrowserRouter entry.
- Moved standalone route ownership into AppRoutes.tsx under /quest5Tier.
- Removed Federation plugin/config and React Router framework scripts/dependencies.

- Removed stale routes.ts because standalone routing now lives in AppRoutes.tsx.
- Updated local env examples so QUEST5TIER_UI points to http://localhost:5185/quest5Tier.
- Added minimal loader error UI to avoid blank screens when Q5 API/config is unavailable.

Verification:

- pnpm run build passed in module/quest5Tier/ui.
- Verified runtime config can load through /env.json locally.
- Verified Q5 Functions query API connectivity with local PostgreSQL after local-only setup.

Notes:

- HomePage was not changed.
- local.settings.json, dist/env.json, reports, and Service Bus emulator config are not included.
- Q5 command flows depend on Service Bus queue processing; local Service Bus setup is being clarified separately.

11. Ponytail review

`deploy/localDev/serviceBus/config.json:L10-L149: delete: Service Bus emulator queue expansion is unrelated to Q5 standalone UI. Revert this file.`

`module/quest5Tier/func/deploy/localDev/serviceBus/queue.json:L3-L142: delete: local Service Bus queue config changes are not part of this PR. Revert this file unless Jake explicitly asks for Service Bus setup.`

`module/profile/func/local.settings.json:L18: delete: profile CORS local setting change. Local-only config, not Q5 standalone code.`

`net: -148 lines possible.`

12. 다음 단계

1. PR에 들어가면 안 되는 3개 파일을 원복한다.

```
```text
deploy/localDev/serviceBus/config.json
module/quest5Tier/func/deploy/localDev/serviceBus/queue.json
module/profile/func/local.settings.json
```
```

2. `git status --short`로 PR 파일 목록을 다시 확인한다.
3. `module/quest5Tier/ui`에서 `pnpm run build`를 다시 실행한다.
4. PR 전에 변경 파일 목록을 사용자에게 다시 보여주고 확인받는다.